

PHASE 2 – REQUIREMENT ANALYSIS

Project: ComicCraft – AI Comic Story Creator using Gemini Models

1. Introduction

Requirement Analysis is the second phase of the ComicCraft project. In this phase, we identify what the system needs to do, what users need, and what technologies and resources are required.

ComicCraft is an AI-based web application that converts a user's simple story idea into an organized comic story with characters, scenes, dialogues, and panel descriptions.

2. Objectives of Phase 2

The main objectives are:

1. Identify user requirements.
2. Identify functional requirements.
3. Identify non-functional requirements.
4. Identify hardware and software requirements.
5. Define the inputs and outputs of the system.
6. Define the basic system modules.
7. Prepare the requirements for the development phase.

3. Functional Requirements

The system should provide the following functions:

FR1 – Enter Story Prompt

The user can enter a simple story idea.

Example:

A young girl discovers a magical book.

FR2 – Generate Comic Story

The system processes the user's prompt and generates a comic story using Gemini AI.

FR3 – Generate Characters

The system identifies and describes the main characters.

FR4 – Generate Scenes

The system creates suitable scenes based on the story.

FR5 – Generate Dialogues

The system generates dialogues for the characters.

FR6 – Generate Comic Panels

The story is divided into multiple comic panels.

FR7 – Generate Image Prompts

The system creates prompts that can be used for generating comic illustrations.

FR8 – Display Result

The generated story and comic panels are displayed to the user.

FR9 – Download Comic

The completed comic can later be downloaded by the user.

4. Non-Functional Requirements

1. Performance

The application should generate results within a reasonable amount of time.

2. Usability

The interface should be simple and easy to use.

3. Reliability

The system should handle invalid or empty prompts properly.

4. Security

API keys should not be exposed in the frontend.

5. Scalability

The application should be designed so additional AI and image-generation features can be added later.

6. Maintainability

The frontend, backend, and AI components should be organized separately.

5. User Requirements

The user should be able to:

- Enter a story idea.
- Generate a comic story.
- View characters.
- View scenes.
- View dialogues.
- View comic panels.
- Generate or use image prompts.
- View the final comic.
- Download the comic.

6. System Requirements

Software Requirements

Component	Technology
Frontend	HTML, CSS, JavaScript
Backend	Python
API Framework	FastAPI
AI	Google Gemini
Image Generation	Gemini/image-generation model or another image API
Browser	Chrome / Edge / Firefox
Code Editor	VS Code
API Testing	Postman / Browser

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Internet connection
- Modern web browser
- Keyboard and mouse

7. Input and Output

Input

The user provides:

Story Prompt

Example:

A young girl discovers a magical book.

Output

The system generates:

- Story
- Characters
- Scenes
- Dialogues
- Comic Panels
- Image Prompts

8. System Modules

Module 1 – User Interface

Allows the user to enter the story prompt.

Module 2 – Story Generation

Sends the prompt to the backend and generates the story.

Module 3 – Character Generation

Creates character descriptions.

Module 4 – Scene Generation

Creates descriptions of the scenes.

Module 5 – Panel Generation

Divides the story into comic panels.

Module 6 – Image Prompt Generation

Creates prompts for comic illustrations.

Module 7 – Comic Output

Displays the final generated comic.

9. Requirement Analysis Workflow

User

↓

Enter Story Prompt

↓

Frontend

↓

FastAPI Backend

↓
Gemini AI
↓
Story + Characters + Scenes + Dialogues
↓
Comic Panel Generation
↓
Image Prompts
↓
Final Comic

10. Phase 2 Coding – Requirement Prototype

main.py

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(
    title="ComicCraft",
    description="AI Comic Story Creator using Gemini Models",
    version="1.0"
)

class StoryRequest(BaseModel):
    prompt: str

@app.get("/")
def home():
    return {
        "project": "ComicCraft",
        "phase": "Phase 2 - Requirement Analysis",
        "message": "ComicCraft API is running"
    }

@app.post("/analyze-requirement")
def analyze_requirement(request: StoryRequest):

    prompt = request.prompt.strip()

    if not prompt:
        return {
            "success": False,
            "message": "Please enter a story prompt."
        }

    return {
        "success": True,
        "prompt": prompt,
        "requirements": {
            "story": True,
            "characters": True,
```

```

        "scenes": True,
        "dialogues": True,
        "comic_panels": True,
        "image_prompts": True
    }
}

```

What this code does

- Creates a FastAPI application.
- Accepts a story prompt.
- Checks whether the prompt is empty.
- Identifies the required ComicCraft components.
- Returns the requirement information as JSON.
- Gemini API integration can be added in the next development phase.

11. Phase 2 Frontend Coding

index.html

```

<!DOCTYPE html>
<html>

<head>
    <title>ComicCraft - Requirement Analysis</title>
</head>

<body>

    <h1>ComicCraft</h1>
    <h3>AI Comic Story Creator</h3>

    <p>Enter your comic story idea:</p>

    <textarea
        id="prompt"
        rows="6"
        cols="50"
        placeholder="Example: A young girl discovers a magical book">
    </textarea>

    <br><br>

    <button onclick="analyzeRequirement()">
        Analyze Requirement
    </button>

    <h2>Requirement Analysis Result</h2>

    <pre id="result"></pre>

    <script>

```

```

    async function analyzeRequirement() {

        const prompt =
            document.getElementById("prompt").value;

        if (prompt.trim() === "") {
            document.getElementById("result").textContent =
                "Please enter a story idea.";
            return;
        }

        const response = await fetch(
            "http://127.0.0.1:8000/analyze-requirement",
            {
                method: "POST",

                headers: {
                    "Content-Type": "application/json"
                },

                body: JSON.stringify({
                    prompt: prompt
                })
            }
        );

        const data = await response.json();

        document.getElementById("result").textContent =
            JSON.stringify(data, null, 2);
    }

</script>

</body>

</html>

```

12. Expected Output

Input

A young girl discovers a magical book.

Output

```

{
  "success": true,
  "prompt": "A young girl discovers a magical book.",
  "requirements": {
    "story": true,
    "characters": true,
    "scenes": true,
    "dialogues": true,

```

```
        "comic_panels": true,  
        "image_prompts": true  
    }  
}
```

13. Phase 2 Deliverables

- User requirements identified
- Functional requirements identified
- Non-functional requirements identified
- Hardware requirements identified
- Software requirements identified
- System modules identified
- Input and output identified
- System workflow prepared
- Basic requirement-analysis prototype developed
- Frontend and backend requirement prototype prepared

Phase 2 Conclusion

The Requirement Analysis Phase defines the requirements and functions of ComicCraft. The system will accept a user's story idea and use AI to generate stories, characters, scenes, dialogues, comic panels, and image prompts. These requirements will be used as the foundation for Phase 3 – Project Design Phase.